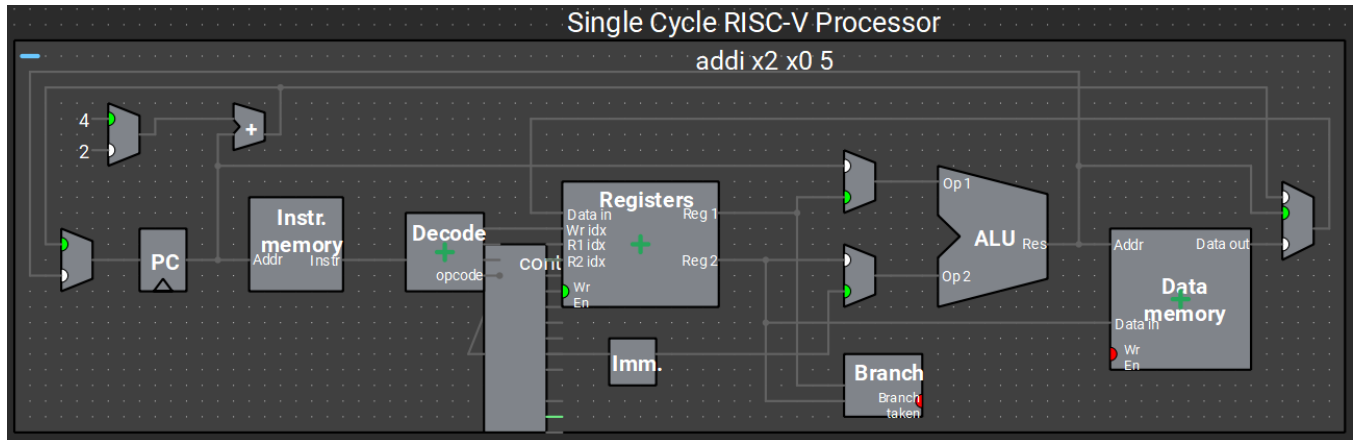


LAB 2: Subprocesor

Date: 11 July of 2025



Proffesor: Dr. Francisco Javier Rodríguez Navarrete

Students: Cesar Eduardo Inda Cenicerros

Alonso Emmanuel López Macías

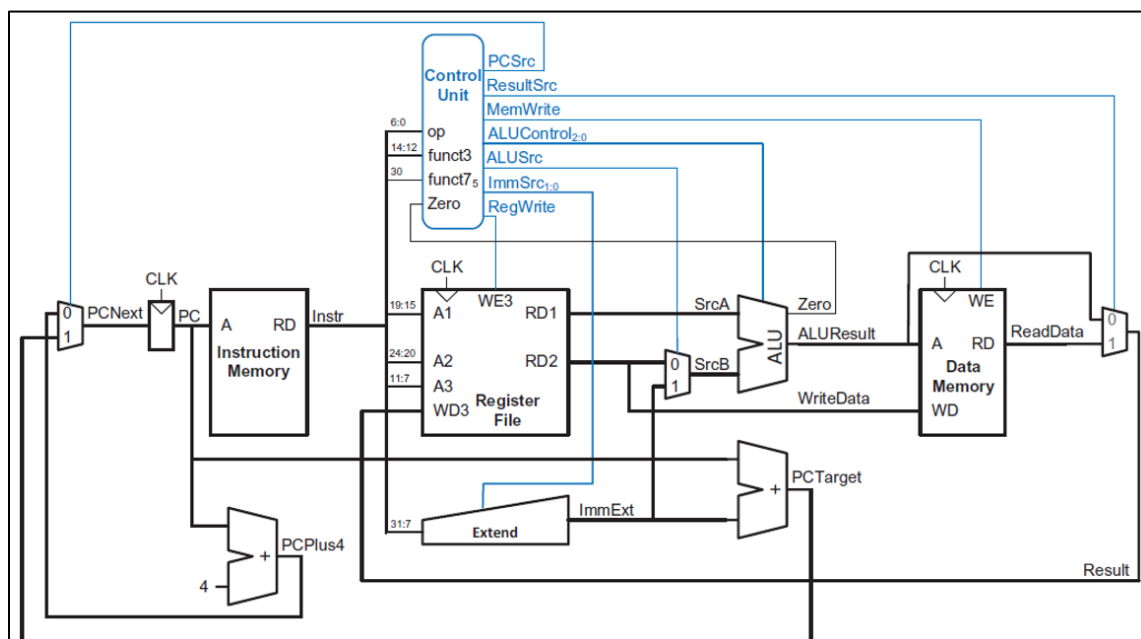
Team: 14

Introduction

Before this laboratory, we had a classroom session where we were introduced to classical blocks, the data path and control path, as well as the integration of these blocks. We covered the importance of the "**single-cycle processor**," how it works, and some RISC-V instructions, including the types of instructions and their operations. We also explored examples of how programs behave based on these fields or internal preconfigurations of the processor's microarchitecture, that is, each instruction already defined internally, which the user can utilize. To apply this, we created a project in Vivado focused on implementing this single-cycle concept.

An important part of digital circuit design, especially during its development and verification stages—is ensuring a solid design based on the schematic interpretation of a microarchitecture. This is crucial, particularly when working in multidisciplinary teams, where clear and readable communication of ideas is necessary to avoid design errors in early stages. In this laboratory, we based our work on a given microarchitecture to develop the RTL design logic, also taking into account a RISC-V considerations table.

Address	Instruction	Type	Fields						Machine Language
0x1000	L7: lw x6, -4(x9)	I	imm _{11:0}	rs1	f3	rd	op		FFC4A303
0x1004	sw x6, 8(x9)	S	imm _{11:5}	rs2	rs1	f3	imm _{4:0}	op	0064A423
0x1008	or x4, x5, x6	R	funct7	rs2	rs1	f3	rd	op	0062E233
0x100C	beq x4, x4, L7	B	imm _{12:10:5}	rs2	rs1	f3	imm _{4:1,11}	op	FE420AE3



Requirements

- Compile & test the program in **“Vivado”**.
- Debug the TestBench **“test the console and waveform values”**.

Development

We use the microarchitecture interpretation for do the RTL files in this case as following:

PC.SV

```
`timescale 1ns / 1ps

module pc #(
    parameter WIDTH = 32,                // Ancho del bus del contador de programa
    parameter RESET_ADDR = 32'h0000_0000 // Dirección de reinicio del PC
)()
    input wire clk,                      // Señal de reloj
    input wire arstn,                   // Reset asíncrono activo en bajo
    input wire [WIDTH-1:0] PCNext,      // Valor siguiente del PC
    output reg [WIDTH-1:0] PC           // Salida actual del PC
);

    always @(posedge clk or negedge arstn) begin
        if (!arstn) begin
            PC <= RESET_ADDR;
        end else begin
            PC <= PCNext;
        end
    end
endmodule
```

ADDER.SV

```
`timescale 1ns / 1ps

module adder #(parameter WIDTH=32)

    (
        input wire [WIDTH-1:0] a,
        input wire [WIDTH-1:0] b,

        output wire [WIDTH-1:0] sum
    );

    assign sum = a + b;

endmodule
```

ALU.SV

```
`timescale 1ns / 1ps

module alu #(parameter WIDTH = 32) (
    input wire signed [WIDTH-1:0] SrcA,
    input wire signed [WIDTH-1:0] SrcB,
    input wire [2:0] ALUControl,
    input wire ALUfunc7_5,
    output reg signed [WIDTH-1:0] ALUResult
);

    always @(*) begin
        ALUResult = 0;
        case(ALUControl)
            3'b000: ALUResult = (ALUfunc7_5) ? SrcA - SrcB : SrcA + SrcB;
            3'b001: ALUResult = SrcA << $unsigned(SrcB[4:0]);
            3'b010: ALUResult = SrcA < SrcB;
            3'b011: ALUResult = $unsigned(SrcA) < $unsigned(SrcB);
            3'b100: ALUResult = SrcA ^ SrcB;
            3'b101: ALUResult = (ALUfunc7_5) ? SrcA >>> $unsigned(SrcB[4:0]) : SrcA >> $unsigned(SrcB[4:0]);
            3'b110: ALUResult = SrcA | SrcB;
            3'b111: ALUResult = SrcA & SrcB;
        endcase
    end
endmodule
```

CONTROL UNIT.SV

In this case we omitted this part because the code represents more than 100 lines of code.

IMM.SV

```
`timescale 1ns / 1ps

module Imm #(parameter WIDTH = 32) (
    input wire [31:0] Instr,
    input wire [2:0] ImmSrc,
    output reg signed [WIDTH-1:0] ImmExt
);

    always @(*) begin
        case (ImmSrc)
            3'b000: ImmExt = {{20{Instr[31]}}, Instr[31:20]}; // I
            3'b001: ImmExt = {{20{Instr[31]}}, Instr[31:25], Instr[11:7]}; // S
            3'b010: ImmExt = {{20{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8], 1'b0}; // B
            3'b011: ImmExt = {{20{Instr[31]}}, Instr[19:12], Instr[20], Instr[30:21], 1'b0}; // J
            3'b100: ImmExt = {Instr[31:12], 12'h000}; //U
            default: ImmExt = 0;
        endcase
    end
endmodule
```

PARAM_MUX.SV

```
`timescale 1ns / 1ps

module param_mux #(parameter int DATA_WIDTH = 32, parameter int NUM_INPUTS = 4) (
    input wire signed [NUM_INPUTS-1:0][DATA_WIDTH-1:0] in, // Array of inputs
    input wire [$clog2(NUM_INPUTS)-1:0] sel,                // Select signal
    output reg signed [DATA_WIDTH-1:0] out                  // Output
);

    always @(*) begin
        out = '0;
        for (int i = 0; i < NUM_INPUTS; i++) begin
            if (sel == i)
                out = in[i];
        end
    end

endmodule
```

RAM.SV

In this case we omitted this part because the code represents more than 100 lines of code.

REGFILE.SV

```
`timescale 1ns / 1ps

module regfile #(parameter WIDTH = 32) (
    input wire clk,
    input wire [4:0] A1,
    input wire [4:0] A2,
    input wire WE3,
    input wire [4:0] A3,
    input wire [WIDTH-1:0] WD3,
    output reg signed [WIDTH-1:0] RD1,
    output reg signed [WIDTH-1:0] RD2
);

    reg [WIDTH-1:0] X [0:31]; //32 bits para cada registro es decir = 1024
    // RS1
    always @(*) begin
        if(A1 != 0) begin
            RD1 = X[A1];
        end else begin
            RD1 = {{WIDTH}{1'b0}};
        end
    end

    // RS2
    always @(*) begin
        if(A2 != 0) begin
            RD2 = X[A2];
        end else begin
            RD2 = {{WIDTH}{1'b0}};
        end
    end

    // RD
    always @(posedge clk) begin
        if(WE3 && A3 != 0) begin
            X[A3] <= WD3;
        end
    end

endmodule
```

ROM.SV

```
`timescale 1ns / 1ps

module rom #(parameter WIDTH = 32, parameter SIZE = 1024) (
    // R-PORT
    input wire [WIDTH-1:0] A,
    output reg [31:0] RD
);

    reg [7:0] mem [0:(SIZE * 4) - 1]; // SIZE * 4 because each 32-bit word = 4 bytes

    always @(*) begin
        RD = { mem[A + 3], mem[A + 2], mem[A + 1], mem[A] };
    end

    function void init_word(
        input reg [WIDTH-1:0] byte_address,
        input reg [31:0] word_data
    );
        begin
            mem[byte_address + 0] = word_data[7:0];
            mem[byte_address + 1] = word_data[15:8];
            mem[byte_address + 2] = word_data[23:16];
            mem[byte_address + 3] = word_data[31:24];
        end
    endfunction

    initial begin
        void'(init_word(32'h1000, 32'hFFC4A303)); // LW
        void'(init_word(32'h1004, 32'h0064A423)); // SW
        void'(init_word(32'h1008, 32'h0062E233)); // OR
        void'(init_word(32'h100C, 32'hFE420AE3)); // BEQ
    end

endmodule
```

Exist other files considerations for use in the project but some codes are very extensive for reduce the information in the report we simplify with some main files in this recompilation for documentation, but is important to know that exist other files, in this part we reduce some logic for the RISC-V and Testbench sections.

